#12 캠 상태 화면(캠 목록) — 코드 상세 설명서

출처: Notion `[#12] 코드리뷰 · 캠 상태 화면(캠 목록)` (PR #110). 아래 코드는 Notion에 적힌 코드다.

1. 전체 흐름

```
관리자 화면: 캠 목록 요청 (regionId는 선택)
   → CameraController.list
   → CameraListService.list
        ├ regionId 없음 → 삭제 안 된 캠 전부
        └ regionId 있음 → RegionLookupService: 그 지역 아래 동 번호
                        → ZoneService: 그 동들의 구역 번호
                        → 그 구역의 캠
        → ZoneService.findSummaries      구역 요약(이름·면적·동 번호) 한 번에
        → RegionLookupService.findPaths  동들의 지역 경로 한 번에
   → 200 목록
```

2. 명세가 요구한 것 vs 구현

명세 (FNC-A-022)	구현
GET /api/v1/manage/cameras (regionId가 있으면 그 아래 구역의 캠만)	`CameraController.list`, `CameraListService.list`
구역 정보(구·동 경로, 면적)를 함께 반환, 카드 수만큼 따로 조회하지 않도록 한 번의 쿼리 묶음	`ZoneService.findSummaries` (1회), `RegionLookupService.findPaths` (1회)
최근 1분 통계, 오늘 혼잡·급증 이벤트 수, 최근 10분 통계	**미구현** (stat=null, todayEventCount=0, recent10m=[])
알림 SSE·통계 SSE로 상태·진행률 갱신	이슈가 **범위 밖(2단계)** 으로 정함
준비 중이면 진행률 막대(0~100%)	`progress` 는 준비 중일 때만 값
상태·진행률·오류 내용 표시	`status`, `progress`, `errorMessage` 포함

3. 코드

3-1. 저장소(질문 도구) 추가

```
// CameraRepository (common)

/** 삭제되지 않은 모든 캠을 번호 순으로(FNC-A-022 캠 목록). */
List<Camera> findByDeletedAtIsNullOrderByIdAsc();

/** 주어진 구역들의 삭제되지 않은 캠을 번호 순으로(FNC-A-022 지역 필터). */
List<Camera> findByZoneIdInAndDeletedAtIsNullOrderByIdAsc(Collection<Long> zoneIds);
```

메서드 이름이 곧 질문이다. "삭제 시각이 비어 있는 캠을 번호 순으로 모두" / "이 구역들 안에 있고 삭제 시각이 비어 있는 캠을 번호 순으로".

```
// ZoneRepository (common)

/** 주어진 읍·면·동에 속한 삭제되지 않은 구역 번호(캠 목록 지역 필터, FNC-A-022). */
```

```
@Query("select z.id from Zone z where z.dongId in :dongIds and z.deletedAt is null")
List<Long> findActiveIdsByDongIdIn(@Param("dongIds") Collection<Long> dongIds);
```

```
// RegionRepository (common)

/** 상위 지역(parent)까지 한 번에 불러온다. 지역 경로(시·군·구 · 읍·면·동) 표시에 쓴다. */
@Query("select r from Region r left join fetch r.parent where r.id in :ids")
List<Region> findAllWithParentByIdIn(@Param("ids") Collection<Long> ids);

/** 지역(시·도, 시·군·구, 읍·면·동 중 아무거나) 아래에 있는 읍·면·동 번호를 구한다. */
@Query("""
        select r.id from Region r
        left join r.parent p
        left join p.parent g
        where r.level = :level and (r.id = :regionId or p.id = :regionId or g.id = :regionId)
        """)
List<Long> findIdsOfLevelUnder(@Param("regionId") Long regionId, @Param("level") RegionLevel level);
```

부분	쉬운 뜻
`left join fetch r.parent`	동을 가져올 때 그 위(구) 정보를 **함께** 가져온다. 이걸 안 하면 동마다 "위가 누구지?"를 따로 물어봐서 쿼리가 동 수만큼 늘어난다
`r.id = :regionId or p.id = :regionId or g.id = :regionId`	"내가 그 지역이거나, 내 위가 그 지역이거나, 내 위의 위가 그 지역이면" 포함. 시·도(할아버지), 시·군·구(부모), 읍·면·동(나) 어느 것을 줘도 그 아래 동이 나온다
`left join`을 쓴 이유	위가 없는 행(시·도)이 중간에 빠지지 않게 하려는 것. 지금은 `level = EMD`로 고르니 동만 남는다
`@Query`	메서드 이름만으로 표현하기 어려운 질문을 직접 쓴다 (JPQL)

3-2. 응답 상자: CameraListItem

```
public record CameraListItem(
        Long id, Long zoneId, String zoneName, String regionPath, double areaM2,
        String name, CameraSourceType sourceType, String sourceUri,
        CameraStatus status, Integer progress, String errorMessage, boolean runRequested,
        Stat stat, int todayEventCount, List<RecentCount> recent10m) {

    /** 최근 1분 통계(5분 안에 통계가 없으면 null). */
    public record Stat(LocalDateTime statAt, double countAvg, double density, int level) {}

    /** 최근 10분 1분 통계의 평균 인원 한 점. */
    public record RecentCount(LocalDateTime at, double countAvg) {}
}
```

- 명세의 응답 모양 그대로다. (`zoneName`, `regionPath`, `areaM2`, `stat`, `todayEventCount`, `recent10m` 포함)
- `progress`는 `Integer`(숫자 또는 null)다. 준비 중일 때만 값이 있다.
- `Stat`·`RecentCount`는 상자 모양만 정해 둔 것이다. 통계가 들어오면 이 상자에 담아 내려 보낸다.

3-3. 총무: CameraListService (핵심)

```
@Service
@RequiredArgsConstructor
public class CameraListService {

    private final CameraRepository cameraRepository;
    private final ZoneService zoneService;
    private final RegionLookupService regionLookupService;

    @Transactional(readOnly = true)
    public List<CameraListItem> list(Long regionId) {
        List<Camera> cameras = regionId == null
```

```
                ? cameraRepository.findByDeletedAtIsNullOrderByIdAsc() : findInRegion(regionId);
        if (cameras.isEmpty()) {
            return List.of();
        }

        Set<Long> zoneIds = cameras.stream().map(Camera::getZoneId).collect(Collectors.toSet());
        Map<Long, ZoneSummary> zones = zoneService.findSummaries(zoneIds);
        Set<Long> dongIds = zones.values().stream().map(ZoneSummary::dongId).collect(Collectors.toSet());
        Map<Long, String> regionPaths = regionLookupService.findPaths(dongIds);

        return cameras.stream()
                .map(camera -> toItem(camera, zones.get(camera.getZoneId()), regionPaths))
                .toList();
    }

    private List<Camera> findInRegion(Long regionId) {
        List<Long> dongIds = regionLookupService.findDongIdsUnder(regionId);
        if (dongIds.isEmpty()) { return List.of(); }
        List<Long> zoneIds = zoneService.findActiveZoneIdsInDongs(dongIds);
        if (zoneIds.isEmpty()) { return List.of(); }
        return cameraRepository.findByZoneIdInAndDeletedAtIsNullOrderByIdAsc(zoneIds);
    }

    private CameraListItem toItem(Camera camera, ZoneSummary zone, Map<Long, String> regionPaths) {
        // stat·todayEventCount·recent10m은 1분 통계·이벤트가 들어오면 여기서 채운다(지금은 null · 0 · 빈 목록)
        return new CameraListItem(
                camera.getId(), camera.getZoneId(),
                zone == null ? null : zone.name(),
                zone == null ? null : regionPaths.get(zone.dongId()),
                zone == null ? 0 : zone.areaM2().doubleValue(),
                camera.getName(), camera.getSourceType(), camera.getSourceUri(), camera.getStatus(),
                camera.getStatus() == CameraStatus.PREPARING ? camera.getProgress() : null,
                camera.getErrorMessage(), camera.isRunRequested(),
                null, 0, List.of());
    }
}
```

부분	쉬운 뜻
`regionId == null ? … : findInRegion(regionId)`	지역을 안 고르면 모든 캠, 고르면 그 지역 안의 캠만
`zoneService.findSummaries(zoneIds)`	캠들의 구역을 **한 번에** 물어봐서 이름·면적·동 번호를 받는다
`regionLookupService.findPaths(dongIds)`	동들의 지역 경로를 **한 번에** 받는다 ("종로구 · 삼청동")
`ZoneService`·`RegionLookupService`를 거친다	구역·지역의 저장소를 캠 쪽에서 직접 쓰지 않고 그 영역의 공개 서비스만 부른다 (팀 규칙)
`PREPARING ? progress : null`	준비 중일 때만 진행률을 준다. 내 해석이다
`null, 0, List.of()`	통계·이벤트 칸은 지금 비워 둔다. 데이터가 생기면 여기만 채우면 된다
`if (dongIds.isEmpty()) return List.of()`	중간에 비면 더 묻지 않고 바로 빈 목록

3-4. 구역·지역 창구

```
// ZoneService (zone 도메인) — 추가된 두 메서드
@Transactional(readOnly = true)
public Map<Long, ZoneSummary> findSummaries(Collection<Long> zoneIds) {
    return zoneRepository.findAllById(zoneIds).stream()
            .map(ZoneSummary::from)
            .collect(Collectors.toMap(ZoneSummary::id, Function.identity()));
}

@Transactional(readOnly = true)
public List<Long> findActiveZoneIdsInDongs(Collection<Long> dongIds) {
    return zoneRepository.findActiveIdsByDongIdIn(dongIds);
```

```
}

// ZoneSummary — 구역 엔티티를 밖으로 내보내지 않고 필요한 값만 담는 상자
public record ZoneSummary(Long id, String name, BigDecimal areaM2, Long dongId) { ... }
```

```
// RegionLookupService (region 도메인)
@Transactional(readOnly = true)
public List<Long> findDongIdsUnder(Long regionId) {
    return regionRepository.findIdsOfLevelUnder(regionId, RegionLevel.EMD);
}

@Transactional(readOnly = true)
public Map<Long, String> findPaths(Collection<Long> dongIds) {
    return regionRepository.findAllWithParentByIdIn(dongIds).stream()
            .collect(Collectors.toMap(Region::getId, this::pathOf, (first, second) -> first));
}

private String pathOf(Region dong) {
    Region parent = dong.getParent();
    return parent == null ? dong.getName() : parent.getName() + " · " + dong.getName();
}
```

- 지역 경로는 시·군·구 이름 + " · " + 읍·면·동 이름이다. (예: "종로구 · 삼청동") 명세 예시 "유성구 · 궁동"과 같은 형태다. 시·도는 뺀다.
- `(first, second) -> first` : 혹시 같은 번호가 두 번 나와도 오류 없이 첫 번째를 쓴다. (방어)
- `ZoneSummary`로 넘기는 이유: 엔티티를 도메인 밖으로 내보내면 나중에 필드를 바꿀 때 여러 곳이 깨진다.

3-5. 경비원: CameraController에 추가된 메서드

```
@GetMapping
public List<CameraListItem> list(@RequestParam(required = false) Long regionId) {
    return cameraListService.list(regionId);
}
```

`GET /api/v1/manage/cameras` (관리자 앱은 앞에 `/admin`). `regionId`는 선택이다. 없어도 된다.

4. 테스트 9개 (전부 통과)

테스트	확인하는 것
`list_noCameras_empty`	캠이 없으면 빈 목록(200)
`list_returnsCameraWithZoneAndRegionInfo`	캠 정보와 구역 이름·지역 경로(종로구 · 삼청동)·면적·상태 STOPPED
`list_progressOnlyWhilePreparing_andErrorMessage`	준비 중 40% → 40, 동작 중·오류 → null, 오류 내용 표시
`list_excludesDeleted_andOrdersById`	삭제된 캠은 빠지고 번호 순
`list_statAndEventsNotYetConnected`	stat=null, todayEventCount=0, recent10m=[] (현재 상태를 테스트로 고정. 데이터가 연결되면 이 테스트를 고친다)
`list_filterBySigungu`	종로구 → 삼청동·사직동 캠만, 중구 → 소공동 캠만
`list_filterByEmdAndSido`	삼청동(읍·면·동) → 그 동 캠만, 서울특별시(시·도) → 3개 전부
`list_filterNoMatch_empty`	없는 regionId, 캠 없는 동 → 빈 목록(200)
`list_queryCountDoesNotGrowWithCameraCount`	캠 1개일 때와 3개일 때 실행된 SQL 수가 같다 (하이버네이트 통계로 N+1 방지 확인)

5. 왜 이렇게 만든 게 맞는가

결정	이유	근거
카메라·구역·지역 경로를 각각 한 번의 쿼리로 모으고 이어 붙임	카드 수만큼 따로 조회하면 캠이 늘수록 느려짐. 설계 단계에서 막음	FNC-A-022 처리 순서 2
구역·지역 조회는 그 영역의 공개 서비스를 거침	다른 도메인의 엔티티·repository를 직접 쓰지 않음	package-structure.md
구역 엔티티 대신 `ZoneSummary` 상자를 넘김	엔티티를 밖으로 내보내면 나중에 필드를 바꿀 때 여러 곳이 깨짐	package-structure.md
응답 상자에 통계·이벤트 칸을 미리 두고 값만 비움	화면이 같은 모양의 응답을 계속 받고, 데이터만 나중에 채우면 된다	api/public.md 응답 예시
페이지를 나누지 않음	구역당 캠 1대라 캠 수가 적고 명세가 페이지를 정하지 않음	api/public.md

6. 선배가 물을 만한 질문과 답

질문	답
"통계 칸이 왜 비어 있어?"	통계를 만드는 코드(#18 PR #102, 혼잡 단계 #19 PR #104)와 이벤트 테이블이 dev에 없다. 사용자와 상의해 "있는 것만 먼저 구현"을 선택했다. 칸은 응답에 미리 두어서 데이터가 생기면 `toItem`만 고치면 된다.
"N+1이 안 생기는 걸 어떻게 알아?"	하이버네이트 통계로 실행된 SQL 수를 세는 테스트가 있다. 캠 1개일 때와 3개일 때 같은 수다. 0이 아니라는 것도 확인해서 "세지 못한 것"이 아님을 확인했다.
"구역 저장소를 직접 쓰면 더 간단하잖아?"	다른 도메인의 repository를 직접 쓰지 말라는 팀 규칙이다. 그 영역이 공개한 서비스만 부른다.
"`ZoneSummary`는 왜 따로 만들어?"	엔티티를 도메인 밖으로 내보내면 필드를 바꿀 때 여러 곳이 깨진다. 필요한 값만 담아 넘긴다.
"`progress`를 왜 준비 중에만 줘?"	명세 예시에서 RUNNING이 null이고 "준비 중이면 막대"라서 합쳐 해석했다. 정확히 정해진 건 아니라서 화면 담당과 맞춰야 한다. 바꾸는 건 한 줄이다.
"없는 지역을 주면 왜 404가 아니라 빈 목록이야?"	명세에 따로 없고, 화면이 없는 지역을 보내는 일이 드물어 빈 목록이 단순하다. 엄격하게 하려면 404를 명세에 추가하자고 제안할 수 있다.
"실시간 갱신은?"	이슈가 2단계(SSE)로 미뤘다. 이 PR은 목록을 한 번 읽어 주는 1단계다.

7. 명세 대조 (FNC-A-022)

명세 항목	구현	확인
GET /api/v1/manage/cameras (regionId 선택)	`CameraController.list`	`list_filterBySigungu`, `list_filterByEmdAndSido`, `list_filterNoMatch_empty`
구역 정보를 한 번의 쿼리 묶음으로	`findSummaries`·`findPaths`	`list_queryCountDoesNotGrowWithCameraCount`
최근 1분 통계·오늘 이벤트 수·최근 10분 통계	**미구현** (null/0/빈 목록)	`list_statAndEventsNotYetConnected`
준비 중이면 진행률, 상태·진행률·오류 내용 표시	`status`·`progress`·`errorMessage`	`list_progressOnlyWhilePreparing_andErrorMessage`

명세 항목	구현	확인
알림 SSE·통계 SSE 갱신	이슈 범위 밖(2단계)	-